2022 非专业级别软件能力认证第一轮

（CSP）C++语言模拟试题

考生注意事项:

- 试题纸共有 10 页，答题纸共有 1 页，满分 100 分。请在答题纸上作答，写在试题纸上的一律无效。
- 不得使用任何电子设备（如计算器、手机、电子词典等）或查阅任何书籍资料。

一、单项选择题（共 15 题，每题 2 分，共计 30 分；每题有且仅有一个正确选项）

1. 广域网和局域网是依照（　　）来分的。

   A、网络使用者　　B、信息交换方式　　C、网络连接距离　　D、传输控制规程

2. 在微机中，1K 等于（　　）。

   A、2^{10}　　B、2^{20}　　C、2^{30}　　D、2^{40}

3. 设有编号为 1、2、3、4、5 的五个茶杯和编号为 1、2、3、4、5 的五个杯盖，将五个杯盖盖在五个茶杯上，**至少**有两个杯盖和茶杯的编号相同的盖法有（　　）。

   A、30 种　　B、31 种　　C、32 种　　D、36 种

4. 在**有向图** G 的拓扑序列中，若顶点 Vi 在顶点 Vj 之前，则下列情形**不可能**出现的是（　　）。

   A、G 中有弧<Vi,Vj>

   B、G 中有一条从 Vi 到 Vj 的路径

   C、G 中没有弧<Vi,Vj>

   D、G 中有一条从 Vj 到 Vi 的路径

5. 有一个查找算法，如下：

```
search:
  i=0;
  for(;a[i]!=x && i<n;i++);
  if(i==n)return -1;
      else return i;
```

   其**时间复杂度**为（　　）。

   A、$O(n)$　　B、$O(n\log n)$　　C、$O(n^{\frac{1}{2}})$　　D、$O(n\sqrt{n})$

6. 有如下函数：

```
int func(int n)
{
    int i=0,sum=0;
    while(sum<n)sum+=++i;
    return i;
}
```

   其**时间复杂度**为（　　）。

A、$O(n)$　　B、$O(n\log n)$　　C、$O(n^{\frac{1}{2}})$　　D、$O(n\sqrt{n})$

7. 有如下函数：

```
int fun(int n)
{
    int x=100;
    if(n==x)return n;
        else return fun(++n);
}
```

其**空间复杂度**为（　　）。

A、$O(n)$　　B、$O(n\log n)$　　C、$O(n^{\frac{1}{2}})$　　D、$O(n\sqrt{n})$

8. （　　）首次提出了**计算机科学理论**。

A、冯·诺依曼　　B、图灵　　C、阿达·洛芙莱斯　　D、姚期智

9. 有 6 个信和 6 个信封，则所有信**装错**了信封可能情况总数为（　　）。

A、14833　　B、265　　C、44　　D、1854

10. 有一表达式：(10+20/2*3)/2+8，则其的**后缀**表达式为（　　）。

A、10 20 2 * + 3 / 2 / + 8

B、10 20 / 2 3 * + 2 / 8 +

C、10 20 2 / 3 * + 2 / 8 +

D、10 2 20 / 3 * + 2 + 8 /

11. 用斐波那契堆来实现最小队列的 Prime 算法的时间复杂度为（　　）。

其中，V 为顶点集合，E 为边集合。

A、$O(E\lg V)$　　B、$O(V)$　　C、$O(E+V)$　　D、$O(E+V\lg V)$

12. 设 x=true，y=false，z=true。以下逻辑运算表达式值为 true 的是（　　）。

A、$(\neg x \vee y) \wedge z$

B、$(y \wedge z) \vee (x \wedge y)$

C、$(x \wedge y) \wedge z$

D、$(x \wedge z) \vee y$

13. 已知完全二叉树第 6 层上有 10 个叶子结点，则这棵二叉树的结点总数**最多**是（　　）。

A、108　　B、107　　C、106　　D、105

14. 在一次体育课上，有 100 个学生，每个人有一个号码，从 1 到 100，站成一排。老师出示了一个正整数，然后每人用这个数除以自己的号码，如果整除则蹲下，否则站着。每个人都计算得准确无误，完毕后，老师发现只有两个人站着。则这两个人编号之和的**最大值**为（　　）。

A、24　　B、97　　C、190　　D、93

15. Linux 下，在使用 mkdir 命令创建新的目录时,在其父目录**不存在**时先创建父目录的选项是（　　）。

A、-m　　B、-d　　C、-f　　D、-p

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确填√,错误填×;除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

1.

```
01 #include<iostream>
02 #include<cstdio>
03 using namespace std;
04
05 int n;
06 int a[105],b[105];
07
08 int main()
09 {
10     scanf("%d",&n);
11     for(int i=1;i<=n;++i) scanf("%d",&a[i]),b[i]=(b[i-1]^a[i]);
12     int ans=0;
13     for(int i=1;i<=n;++i) {
14         for(int j=i;j<=n;++j) {
15             ans=max(ans,b[j]^b[i-1]);
16         }
17     }
18     printf("%d\n",ans);
19     return 0;
20 }
```

假设输入的序列长度不会超过 100，且序列内的元素是正整数。完成下面的判断题和单选题：

- 判断题

1) 若将第 12 行放到第 4 行，程序的运行结果**不会**发生改变。（　　）

2) 当输入 5 1 2 1 1 2 时，输出结果为 4。（　　）

3) 若将第 1 行删去，程序**不会**编译错误。（　　）

4) 将 12 行中 ans 的类型声明从 int 改为 double，程序**会**编译错误。（　　）

- 单选题

5) **（2 分）**若输入 7 8 4 1 3 6 5 4，则输出（　　）。

A、12　　B、13　　C、14　　D、15

6) 这段代码的含义为（　　）。

A、求一段序列中异或和最大的连续字段和

B、求一段序列中异或和最大的字段和
C、求一段序列中异或和最大的字段的长度
D、求一段序列中异或和最大的连续字段和的长度

2.

```
01 #include <iostream>
02 #include <cstdio>
03 #include <vector>
04 using namespace std;
05
06 const int maxn = 10000 + 10;
07
08 int n, cnt, d[maxn];
09 vector<int> G[maxn];
10
11 void dfs(int cur, int fat){
12     for(int i = 0;i < G[cur].size();++i){
13
14         if(nxt == fat) continue;
15         d[nxt] = d[cur] + 1;
16         if(d[nxt] > d[cnt]) cnt = nxt;
17         dfs(nxt, cur);
18     }
19 }
20
21 int main(){
22     ios::sync_with_stdio(false);
23     cin>>n;
24     for(int i = 1;i < n;++i){
25         int x, y;
26         cin>>x>>y;
27         G[x].push_back(y);
28         G[y].push_back(x);
29     }
30     dfs(1, 0);
31     d[cnt] = 0;
32     dfs(cnt, 0);
33     cout<<d[cnt]<<endl;
34     return 0;
35 }
```

- 判断题

1) 若将第 22 中的 false 改为 true，程序的结果**不会**改变。(　　)

2) （2 分）若程序输入 3 1 2 2 3，则输出 1。（　　）

3) 若将 14 行删去，程序仍能正常运行。（　　）

- 单选题

4) 该代码的时间复杂度为（　　）。

A、$O(2^n)$　　B、$O(n)$　　C、$O(n!)$　　D、$O(n^2)$

5) 若输入 5 1 2 1 3 3 4 3 5，则输出（　　）。

A、2　　B、3　　C、4　　D、5

6) （4 分）若输出结果为 4，则下面（　　）是输入数据。

A、10 4 5 9 10 1 2 4 6 3 4 4 8 6 7 7 8 8 9

B、8 1 3 2 3 3 4 4 5 4 6 4 7 7 8

C、6 1 2 2 3 3 4 4 5 5 6

D、10 1 9 5 6 10 5 2 6 10 1 5 3 8 3 4 10 7 10

3.

```
01 #include<cstdio>
02 #include<algorithm>
03 using std::max;
04 const int MAXN = 1000 + 6;
05 int n,A[MAXN];
06
07 bool dfs(int d,int maxd){
08     if(A[d] == n) return true;
09     if(d == maxd) return false;
10     int maxv = A[0];
11     for(int i = 1;i <= d;i++) maxv = max(maxv,A[i]);
12     if((maxv << (maxd - d)) < n) return false;
13
14     for(int i = d;i >= 0;i--){
15         A[d + 1] = A[d] + A[i];
16         if(dfs(d + 1,maxd)) return true;
17         A[d + 1] = A[d] - A[i];
18         if(dfs(d + 1,maxd)) return true;
19     }
20     return false;
21 }
22
23 int solve(int m){
24     if(m == 1) return 0;
25     A[0] = 1;
26     for(int maxd = 1;maxd < 13;maxd++) if(dfs(0,maxd)) return maxd;
27     return 13;
```

```
28 }
29
30 int main(){
31     scanf("%d",&n);
32     printf("%d\n",solve(n));
33 }
```

- 判断题

  1) 此程序用 C++98 编译会编译错误。(　　)

  2) **（2 分）**若程序输入 437，则输出 11。(　　)

- 单选题

  3) **（3.5 分）**若程序输入 8764，则输出（　　）。

  A、11　　B、12　　C、13　　D、14

  4) **（4 分）**该程序的 n 不能等于（　　），否则运行时间就会超过 1 秒。

  A、-1　　B、0　　C、-1000　　D、2147483647

  5) 若 n=437，将第 26 行的 maxd 的取值范围由[1,13)改为（　　）时，运行时间会超过 1 秒。

  A、[50,100]　　B、[1,100]　　C、[100,10000]　　D、[-1,100]

三、完善程序（单选题，每小题 3 分，共计 30 分）

1. **（新旧物业）**给出 A 和 B，请你求出从 A 到 B 之间的所有数中，有多少数转化为 2 进制后，只有一个零。其中 $1 \le a,b \le 10^{18}$。

   提示：考虑数位 DP。

```
#include <bits/stdc++.h>
using namespace std;
long long l, r;
long long a[100005];
long long cnt = 0;
long long cntl, cntr;
long long dp[1005][1005][3];
long long dfs(long long x, long long same,int zero, long long tot) {
    if (tot > 1) {
        return 0;
    }
    if (x == 0) {
        if (tot == 1) {
            return 1;
        } else {
            return 0;
        }
```

```
    }
    if (!same && dp[x][tot][zero]!=-1) {
        return ①;
    }
    long long ans = 0;
    long long up = same ? a[x] : 1;
    for (long long i = 0; i <= up; i++) {
        if (i == 0) {
           if(zero>0)
           {
               ans += ②;
           }
           else
           {
               ans += ③;
            }

        } else {
            ans += ④;
        }
    }
    if (!same) {
        dp[x][tot][zero] = ans;
    }
    return ans;
}
int main() {
    scanf("%lld %lld", &l, &r);
    memset(dp,-1,sizeof(dp));
    long long t = r;
    while (t) {
        a[++cnt] = t & 1;
        t >>= 1;
    }
    cntr = dfs(cnt, 1,0, 0);
    t = l - 1;
    cnt = 0;
    while (t) {
        a[++cnt] = t & 1;
        t >>= 1;
    }
    cntl = ⑤;
    printf("%lld", cntr - cntl);
}
```

1) ① 处应填（　　）。
   A、dp[x-1][tot][zero]
   B、dp[x+1][tot][zero]
   C、dp[x][tot][zero]
   D、dp[x&1][tot][zero]
2) ② 处应填（　　）。
   A、dfs(x - 1, same && (i == a[x]),0, tot)
   B、dfs(x - 1, same && (i == a[x]),1, tot)
   C、dfs(x + 1, same && (i == a[x]),1, tot + 1)
   D、dfs(x - 1, same && (i == a[x]),1, tot + 1)
3) ③ 处应填（　　）。
   A、dfs(x - 1, same && (i == a[x]),0, tot)
   B、dfs(x - 1, same && (i == a[x]),1, tot)
   C、dfs(x + 1, same && (i == a[x]),1, tot + 1)
   D、dfs(x - 1, same && (i == a[x]),1, tot + 1)
4) ④ 处应填（　　）。
   A、dfs(x - 1, same && (i == a[x]),0, tot)
   B、dfs(x - 1, same && (i == a[x]),1, tot)
   C、dfs(x + 1, same && (i == a[x]),1, tot + 1)
   D、dfs(x - 1, same && (i == a[x]),1, tot + 1)
5) ⑤ 处应填（　　）。
   A、dfs(cnt,0,0,0)
   B、dfs(cnt,1,0,0)
   C、dfs(cnt,1,1,0)
   D、dfs(cnt,1,0,1)

2. （数字）给你两个数 x，op。分以下两种情况：
若 op=0，则需要输出一个 y，满足 y≥x 且 y 由同一个数字组成。（比如 y=77777）
若 op=1，则在 op=0 的条件上，增加一个 y 可以多一个不同数字。（比如 y=77727）
其中$1 \le x \le 10^{17}$。

```
#include <bits/stdc++.h>
using namespace std;

long long num(int len, int d, int pos, int d2) {
    long long res = 0;
    for (int i = 0; i < len; i++) {
        if (i == pos) {
            res = ①;
        } else {
            res = ②;
        }
    }
    return res;
```

```
}

int main() {
    long long x;
    cin >> x;
    int k;
    cin >> k;

    long long ans = -1;
    if (k == 0) {
        for (int i = 1; i <= 18; i++) {
            for (int d = 1; d < 10; d++) {
                long long y = ③;
                if (y >= x && (ans == -1 || y < ans)) {
                    ans = y;
                }
            }
        }
    } else {
        for (int i = 1; i <= 18; i++) {
            for (int d = 0; d < 10; d++) {
                for (int p = 0; p < i; p++) {
                    for (int d2 = 0; d2 < 10; d2++) {
                        long long y = ④;
                        if (y >= x && (⑤ || y < ans)) {
                            ans = y;
                        }
                    }
                }
            }
        }
    }

    cout << ans << endl;
}
```

1) ① 处应填（　　）。
   A、res + d2
   B、res * 10 + d2
   C、res * 10 + d
   D、res + d
2) ② 处应填（　　）。
   A、res + d2
   B、res * 10 + d2

C、res * 10 + d

D、res + d

3) ③ 处应填（　　）。

A、num(i+1, d, 0, d)

B、num(i-1, d, 0, d)

C、num(i, d, 0, d)

D、num(i, d, 1, d)

4) ④ 处应填（　　）。

A、num(i+1, d, p, d2)

B、num(i-1, d, p, d2)

C、num(i, d, 0, d2)

D、num(i, d, p, d2)

5) ⑤ 处应填（　　）。

A、ans == 0

B、ans == -1

C、ans > 0

D、ans >= 0